



UNIVERSITÀ DEGLI STUDI DI FIRENZE

Bachelor's Degree in Computer Science

CODE SUMMARIZATION IN PYTHON

Project report for the course:

MACHINE LEARNING FOR SOFTWARE ANALYSIS

Student:

Sean Andreini

Student ID:

7133800

Email:

sean.andreini@edu.unifi.it

Professor:

Prof. Fabio Pinelli

Academic Year 2025/2026

Contents

1	INTRODUCTION AND OBJECTIVES	2
2	METHODOLOGY	2
2.1	Data	2
2.2	Models	3
2.2.1	Model 01: LSTM Seq2Seq without Attention and Basic Tokenization	3
2.2.2	Model 02: LSTM Seq2Seq without Attention and AST Tokenization	4
2.2.3	Model 03: LSTM Seq2Seq with Attention and AST Tokenization	4
2.2.4	Model 04: LSTM Seq2Seq with Attention, AST and BPE Tokenization	5
2.2.5	Model 05: Transformer	5
3	Final Models	5
3.1	Grid Search	6
3.2	Results	6
3.2.1	Transformers	6
3.2.2	LSTMs	7
4	Conclusions	7

1 INTRODUCTION AND OBJECTIVES

With code summarization we refer to the ability of generating a summary of a piece of code in natural language. In this project, I'm going to develop a couple of models with this function, in this case focusing solely on Python. The instructions to download and run the models can be found in the `README.md` on the GitHub repository¹.

The main objective is finding the best models for code summarization, comparing different choices and implementations. To evaluate the models we will be using three metrics: Cross Validation just as a base, BLEU and RougeL. The reason I chose both is because BLEU (Bilingual Evaluation Understudy) is a bit more strict, as it makes the score go up based on n-grams (group of consecutive words). So, not only the words must be in the correct order, but they should be exactly the same (if it uses reads instead of loads it interprets it as wrong). Rouge-L (Recall-Oriented Understudy for Gisting Evaluation) bases itself on recall and longest common subsequence. This means that it searches the longest sequence of words present in both texts, with the ability of skipping words (maintaining the order of course). This makes it a bit more forgiving, giving us a score of how well the model is doing regarding the general context.

The whole research will be documented in the notebooks found in the `notebooks/` directory. After the research, which will help us understand which models to use, I'll build the final models (one LSTM and one Transformer). I'll then run a grid search on the final architectures so as to find which parameters to use for training. Then I'll train the best transformer and the best Encoder Decoder twice, once on 20k training samples and once on 50k. After that I'll evaluate them, we'll discuss the scores, and inference the models a bit so as to judge by ourselves how it's performing. If you want to evaluate and inference the models yourself, you can find them in the `releases` section on the repository.

Now, before starting I've done some research to see which models to use. I wanted to build a transformer, but I wasn't sure as to which Encoder Decoder. So I found on a paper² that the best combination seems to be an LSTM with a BPE Tokenizer. Therefore you'll find some different tokenization experiments on an LSTM.

2 METHODOLOGY

2.1 Data

The dataset used for this project is the CodeXGLUE dataset³ given by Microsoft, which is a revised version of the SearchCodeNet database. In the project there's just the Python subset of the dataset. There's about 250k training samples, 14k validation samples and 15k test samples. Each sample contains:

- **repo**: the GitHub repository of the sample.
- **path**: the path to the file in the repository.
- **func.name**: the name of the function.
- **original_string**: the code of the function (including docstring) as a string.
- **language**: the language of the sample (Python in this case).
- **code**: same as `original_string`.

¹<https://github.com/seanandreini/code-summarization-mlsa-project>

²https://www.researchgate.net/publication/394590942_How_Different_Tokenization_Algorithms_Impact_LLMs_and_Transformer_Models_for_Binary_Code_Analysis

³<https://github.com/microsoft/CodeXGLUE/tree/main/Code-Text/code-to-text>

- **code_tokens**: the code (without docstring) already tokenized.
- **docstring**: the docstring (summary) of the function.
- **docstring_tokens**: the docstring already tokenized.
- **sha**: the sha hash of the commit.
- **url**: the URL of the file in the repository.
- **partition**: the partition of the dataset (train, valid, test).

You can find additional information about the dataset in the `notebooks/01-dataset-exploration` notebook. Obviously, since the dataset is already divided into training, validation and test sets, I didn't have to do any splitting. Regarding preprocessing, we're gonna explain the different tokenization methods used later on, since in the notebooks I used different approaches and compared them. For embedding I used the `nn.Embedding` layer provided by Pytorch, which maps tokens indices to vectors of the specified embedding dimension (it can be found in the config files in `config/`).

2.2 Models

I implemented different models for code summarization. Here I am going to explain them briefly (the details can be found in the notebooks).

2.2.1 Model 01: LSTM Seq2Seq without Attention and Basic Tokenization

The first model implemented (`notebooks/model01/03-first-model`) is a LSTM seq2seq without attention. I actually didn't tokenize the data myself, but I used the pre-tokenized data found in the dataset: `code_tokens` and `docstring_tokens` (`notebooks/model01/02-tokenization`). Since that was our first model in the notebooks there's some code used to check if the model was working fine before training it, these steps will not be present in the next notebooks.

The architecture is really simple, there's an Encoder and a Decoder. The `EncoderDecoder` class puts these together, defining a method to initialize the outputs and store them for every step. In the forward method it initializes the decoder hidden states with the encoder output (which are the last hidden state and cell). The first input in the decoder is a new tensor with the BOS token. It then starts the loop and decides to use the generated token to predict the next one or to use the actual token of the docstring (teacher forcing). The `collate` function ensures that every docstring ends with the EOS token, and pads the sequences. Obviously the loss (I used the `CrossEntropyLoss`) is created specifying to ignore the PAD token.

I initially trained the model on 5000 lines (1000 for validation), and set it to 100 epochs, but it stopped at epoch 6 because of early stopping (I initially set a low patience of 5). This actually showed us that the model was overfitting immediately, as the validation loss started increasing after the first epoch (while the training loss kept going down). But since in the notebooks I was just testing the models to compare them, I didn't care and actually preferred to save time in training.

After training I evaluated the model on the test set, obtaining a BLEU score of 0.015 and a RougeL score of 0.089, which is really low, but can be used to compare with other models. During the evaluation loop I printed the predicted strings and the expected strings to check the model with our own eyes, and I could see that not only it had some difficulties with the language in general, but the predictions were completely wrong most of the time. But to be fair it could predict some basic keywords, like "return" or "list", which is still something.

Obviously this was expected as the model is really basic, and I trained it on a very small subset. But to compare it without changing too much the next step was to use the same model, but using Abstract Syntax Tree (AST) tokenization.

2.2.2 Model 02: LSTM Seq2Seq without Attention and AST Tokenization

So, as anticipated, the second model is exactly the same as the first one (LSTM Seq2Seq without attention), but this time I changed tokenization: instead of using the pre-tokenized data (or any type of string tokenization), I started representing the data as Abstract Syntax Trees (ASTs). To do this I used the `ast` library. As you can see in the `notebooks/model01/04-ast-tokenization` notebook, after parsing the code string to an AST I linearize it, so as to create a sequence to feed into our encoder decoder. The docstrings received the same type of tokenization as the first model, so I could see the difference changing just how the code is represented.

Now to be precise, the tokenization done in the notebook isn't actually a plain AST, that is because ASTs don't have that much information. So, in the `linearize_ast` function, which linearizes the tree following the *Structure Based Traversal (SBT)*, I append some additional information, like the function, argument and variable names. This ensures that the model has something else to work with apart from the pure AST, otherwise it would probably have a huge disadvantage. However, I then filter the dictionary removing the keys that show up less than 5 times, so as not to make the vocab explode. In fact, this method created a vocab of just under 14k, which is infinitely better than the first one, which was over 1 million lines.

Something important I had to ensure was that the code and docstring were correct: during the AST parsing I encountered some errors, I think that's because some code is written in Python2. Anyway, the `code_to_ast` function returns the parsed tree only if it's possible, otherwise it returns `None`. What initially ended up happening is that during the linearization when the tree was `None` it got converted to an empty list. So the models were seeing an empty code, with a docstring attached, and that would confuse them during training (this problem was present in validation and test too though). So in the notebook you can see that I save the indices of the code strings that get converted successfully, so as to save in the dataset only the code (and related docstrings) that gets a correct AST.

In the next notebook I pasted the same code as the first model, and trained it, this time on the AST data saved in the previous notebook. The hyperparameters were the same, and so was the evaluating method (BLEU and RougeL). We can see that they're almost the same (just 0.01 higher). Now, although the score is still really low, this is expected as this model is still very undertrained and suffers a lot of overfitting. Since there has been a small improvement, and a smaller vocab can increase the performance, we're going to use ASTs from now on.

Now there was just one other question left with this type of model, and that was if some kind of attention could improve its performance.

2.2.3 Model 03: LSTM Seq2Seq with Attention and AST Tokenization

In this model I added attention, scaled-dot product attention to be precise. The attention mechanism is there to take into consideration the whole sequence coming from the encoder, and not just the last element. How that works is by making a product to see how "similar" the query and key are. Since this numbers can become pretty big if the model has a high dimension, it gets scaled, therefore scaled-dot product. The softmax at the end converts this scores into percentages, so the model knows which words have more value when predicting the next. This can be pretty good, especially in generating natural language, so I tried the same encoder decoder with AST and attention.

There was some improvement. The BLEU was almost the same, but the Rouge score actually went up quite a bit (0.05). So this confirms that attention makes the difference.

2.2.4 Model 04: LSTM Seq2Seq with Attention, AST and BPE Tokenization

I explored attention and lack of attention, different code representations, but what about the docstrings? As explained in the dataset exploration the CodeXGLUE gave us an already tokenized dataset, but the tokenization consisted just in splitting on spaces. There's a paper, as I briefly explained in the introduction section, that suggests to use LSTM with Binary Pair Encoding (BPE) tokenization. So I tried that, keeping the same model with attention and AST. I could immediately notice that the training loop was much, much faster. In fact the vocab created was 8k big (a dimension suggested by ChatGPT considering our small training subset).

There's one thing that surprised me though, and that was the scores being actually worse. There's one thing that came to mind to me: with BPE I can see, analyzing the vocab, that words get splitted sometimes. This means that the output generated has more "words" that what is intended. This could influence negatively the BLEU score. What I noticed is that the language was gramatically better, even if the prediction themselves were still far off.

2.2.5 Model 05: Transformer

I wouldn't be making a full research if I didn't try a different model: the transformer. To do this I used the same data as the *model 04*: the AST for code representation and BPE for the docstrings. The model uses cross attention between the encoder and decoder, and self attention on the decoder so it can see what it's saying when it's making the prediction. One thing I've changed is the insertion of the BOS token, or to be more precise: in the encoder decoder model I inserted it when I created the tensor in the forward method. This time, since there's no for loop in the transformer, but I pass the whole sequence, the `Collate Function` puts a BOS token at the beggining of the sentence.

Now obviously since it's a different model the hyperparameters aren't the same, but I tried to replicate the best I could the same environment as the others, so as not to create some type of bias (`d_model` was the same as `embedding_dims`). That being said, after some research and a couple of suggestions by Gemini and ChatGPT for some starting hyperparameters, I put a learning rate of 0.0001 instead of 0.01, because the transformer has some more difficulties with a learning rate too big. In fact, as soon as I changed that parameter, the model continued improving until epoch 8, instead of stopping at epoch 2-3. Oh and since the speed of this model is much better, I increased the `batch_size` to 16.

The scores were a bit higher than model04. Not by much, but they were still better. So, now was the time to build the final models. I chose the models 04 and 05. First of all because I wanted to compare two different architectures, and also because I thought the one with attention, BPE and ASTs was the most "professional" approach.

3 Final Models

As explained above, as the final models I chose the LSTM with attention (model04) and the Transformer (model05), both using AST tokenization for code and BPE tokenization for docstrings (the pretokenized dataset gets saved and shared between the models). The usage of the scripts is explained in details in the `README.md` found in the repository.

3.1 Grid Search

I ran one grid search for model, on 5k samples, to choose the hyperparameters for the final models. First of all I ran the transformer. I put a small batch size (just 4) because my GPU VRAM would run out when training the models with a high `d_model` and `n_layers`. The combinations were: learning rate between 0.0001 and 0.00005, `n_layers` between 4 and 6, and `d_model` between 256 and 128.

After running every combination the grid search results were very promising: there was one model which scored best in every metric: the one with a learning rate of 0.0001, 4 layers and a `d_model` of 256. It scored 0.086 on BLEU, 0.33 on RougeL, and had a validation loss of 6.03.

For the LSTM I ran the same experiment. I actually noticed that I could put a higher batch size (16) without running out of memory. The combinations were: embedding dim between 128 and 256, hidden dim between 256 and 512, number of layers between 1 and 2, learning rate between 0.001 and 0.0005, and a teacher forcing prob between 1.0 and 0.5.

This time the best models were different between metrics, but the scores were kinda the same:

- the one which scored best on BLEU had an embedding dim of 128, an hidden dim of 256, 1 layer, a learning rate of 0.001 and a probability of teacher forcing equal to 1.
- the one which scored best on RougeL had an embedding dim of 128, an hidden dim of 256, 1 layer, a learning rate of 0.0005 and a probability of teacher forcing equal to 1.
- the one which scored best on Cross Validation Loss had an embedding dim of 128, an hidden dim of 256, 2 layers, a learning rate of 0.0005 and a probability of teacher forcing equal to 0.5.

Now, what surprised me is that the LSTM needed a full teacher forcing to achieve the best scores. Anyways, I think the best metric is BLEU, as it is a bit more strict than RougeL, and anyways I could see on the `.csv` that the RougeL scores were very close between those two models, so as the final LSTM I chose the first one. I then proceeded to train for 20k and 50k lines the models I chose.

3.2 Results

First of all I want to say that for predictions I used the multinomial function on the softmax. That is because I didn't want to use greedy predictions (I noticed during the research in the notebooks that it would just output nonsense), and I wanted to explore the general creativity of the model when running inference multiple times.

3.2.1 Transformers

I trained the transformer on a subset of 20k. I then evaluated it and it got a validation loss of 5.58, a BLEU score of 0.11 and a rougeL of 0.35. I then inferenced it with

```
def load_config(path):  
    with open(path, 'r') as f:  
        return json.load(f)
```

and its output was *"Check that the content of the file is a given file ."*. We can see that the scores went up a bit, and the prediction wasn't bad at all. It in fact has gramatical sense, it understands that it's handling a file and opening the content, and it ends with a dot (testing other predictions it seems the model learned that it needs to end the prediction with EOS after the dot).

I trained the same transformer again, this time on a subset of 50k. This time the validation loss was 5.21, but the BLEU score 0.11 and the rougeL 0.35, same as before. What this can tell us is that maybe

the model is learning a bit better the most common terms, which make the validation loss go down, but the semantics are more or less the same.

Obviously I tried to inference it on the same code, and the result was *"get a file using filename"* which was actually great. Now, since we're using multinomial to make our predictions this was achieved after a couple of tries, but the good sign is that almost all the times it gets that it's working with a file.

3.2.2 LSTMs

Let's discuss now of the LSTM trained on 20k samples. The loss was 7.32, the BLEU 0.11 and the rougeL 0.35. We can see that the validation loss is much higher, but the other metrics are exactly the same. I think this could be because of the probabilities being "dispersed": since the loss is based on the logits it's a bit higher because the transformer is more "sure" of its predictions. But since the most probable word is probably still the same, when it makes the prediction it's similar to the transformer.

To confirm this I ran inference multiple times, on the same code as the transformer. The best predicted string was *"s a new task to the data file ."*. But the predictions were very different from one another. It often thought it was handling a dictionary, sometimes it said something about returning a user. This confirms the fact that the logits are probably more similar, so the model is a bit more insecure.

Anyways, after training the model on 50k samples, the results were exactly the same, with the validation loss just 0.02 points lower. What this tells me is that maybe the database isn't diverse enough, our maybe to get better results we would have to increase some hyperparameters if we trained this models in a system that could handle them. The best inferenced result was *"get the path for the given path and the provided format ."*, which is quite similar to the results we got with the other models, but I had the same problem as the other LSTM.

4 Conclusions

Something I want to pay attention to is overfitting. What I noticed is, while both the transformers saved their "best epoch" around epoch 4/5, the LSTMs never had an increase in validation loss since the very first epoch. However, as we saw, the scores were pretty much the same. What this tells us is that the LSTM learns faster than the transformer, which needs a bit more time.

Another particular we should point out is that the transformer stopped getting better after epoch 4/5 as I said, but this wasn't the case when the samples were just 5k (during the grid search). I think this means that with more samples the transformer, training more for epoch, gets to its upper limit faster.

All of this could hint us that maybe we really reached the upper limit of these models with these configurations. I don't think the vocabs are the problem as they're both around 15k words big, so they should have enough ids to represent the most important words.

This was a great experiment, we explored different models and in the final ones we saw how different architectures had some differences, but some things in common too. My conclusion is that for NLP, code summarization in particular in this case, the transformer seems to be the best choice. Yes, the scores were almost the same, but not only the model trained a lot faster, but it was much more precise: running multiple inferences it seemed to guess correctly it was handling a file most of the times, while the LSTMs were guessing a bit more (keep in mind the temperature of the softmax was the same in both models).

Further changes could be increasing the model sizes and training them on the full dataset, in addition to a bigger vocab and maybe a more extensive grid search.